Two-Pointer Pattern

876. Middle of the Linked List **********

Concept

- Use **Slow & Fast pointers**.
- `slow += 1`, `fast += 2`.
- When `fast` reaches end, `slow` is at **middle**.
- If **even length**, returns **2nd middle** (LC requirement).

Edge Cases

- Empty list → `nullptr`.
- Single node → return head.
- Even nodes → returns second middle.
- Odd nodes → exact middle.

Interview Tips

- **Template** used in:
  - Find middle
  - Merge Sort LL
  - Palindrome LL
  - Cycle detection variants
- To get **first middle** (instead of second):

```
while(fast->next && fast->next->next)
```

Code Snippet

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

141. Linked List Cycle**********

Concept

- Use **Slow & Fast pointers (Floyd's Algorithm)**.
- `slow += 1`, `fast += 2`.
- If they **ever meet** → cycle exists.
- If `fast` or `fast->next` becomes `nullptr` → no cycle.

Edge Cases

- Empty list.
- Single node (with/without self-loop).
- Cycle starting at head.
- Long non-cyclic prefix before cycle.

Interview Tips

- **Meeting ⇒ cycle exists**, **not** cycle start.
- Cycle start is asked in **LC 142**.
- Always check:

```
while (fast && fast->next)
```

Code Snippet

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
            return true;
    }

    return false;
}
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

142. Linked List Cycle II

****

Concept

- Step 1: Detect cycle using **Slow & Fast**.
- Step 2: If they meet, move one pointer to `head`.
- Step 3: Move **both one step at a time**.
- The node where they meet again = **cycle start**.

Why it works (Interview Proof)

Let:

- `L` = distance from head to cycle start.
- `C` = cycle length.
- `x` = distance from cycle start to first meeting point.

At first meeting:

```
2 * (L + x) = L + x + k * C
⇒ L = k * C - x
```

So from:

- **Head → cycle start = L**
- **Meeting point → cycle start = C - x (mod C)**

Both pointers need the **same remaining distance**, so they meet exactly at the cycle start.

Edge Cases

- No cycle → return `nullptr`.
- Self-loop.
- Cycle starts at head.
- Long prefix before cycle.

Interview Tips

- **141:** Detect cycle.
- **142:** Find cycle start.
- Most common follow-up: **Explain the math proof** above.

Code Snippet

```
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }

    return nullptr;
}
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

19. Remove Nth Node From End of List

Concept

- Use **Dummy node** (avoids head deletion special case).
- Move `fast` **n+1 steps** ahead.
- Move `fast` & `slow` together until `fast == nullptr`.
- `slow` now points to **node before target** → delete `slow->next`.

Edge Cases

- Remove head (`n == length`) ☑ (dummy handles it).
- Single node.
- Remove last node (`n == 1`).
- Two-node list.

Interview Tips

- **Always use a dummy node**.
- Gap should be `n+1`, not `n`, so `slow` lands on the previous node.
- Don't forget:

```
slow->next = slow->next->next;
```

- Pattern: **Maintain a fixed gap** between two pointers.

Code Snippet

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode *dummy = new ListNode();
    dummy->next = head;

    ListNode *slow = &dummy, *fast = &dummy;

    for (int i = 0; i <= n; i++)
        fast = fast->next;

    while (fast) {
        slow = slow->next;
        fast = fast->next;
    }

    slow->next = slow->next->next;

    return dummy_>next;
}
```

Complexity

- **Time:** `O(n)`

- **Space:** `O(1)`

143. Reorder List

Concept

- **3-step template:**
    a. Find **middle** (Slow & Fast).
    b. **Reverse** second half.
    c. **Alternate merge** both halves.
- Final order:

```
L0 → Ln → L1 → Ln-1 → ...
```

Edge Cases

- Empty / single node.
- Two nodes (no change).
- Odd length → first half has one extra node.
- **Break the list** before reversing:

```
slow->next = nullptr;
```

Interview Tips

- Classic combination of **3 linked list patterns**:
    - Find Middle (876)
    - Reverse List (206)
    - Merge Two Lists (alternate merge)
- Forgetting `slow->next = nullptr` can create a cycle.
- Merge carefully by storing next pointers first.

Code Snippet

```
void reorderList(ListNode* head) {
    if (!head || !head->next) return;

    // Find middle
    ListNode *slow = head, *fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Reverse second half
    ListNode *cur = slow->next;
    slow->next = nullptr;

    ListNode *prev = nullptr;
    while (cur) {
        ListNode *nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }

    // Merge alternately
    ListNode *l1 = head, *l2 = prev;
    while (l2) {
        ListNode *n1 = l1->next;
        ListNode *n2 = l2->next;

        l1->next = l2;
        l2->next = n1;

        l1 = n1;
        l2 = n2;
    }
}
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

Pattern to Remember

```
Find Middle
      ↓
Reverse Second Half
      ↓
Alternate Merge
```

This exact pipeline is reused in several advanced linked list problems.

61. Rotate List **********

Concept

- Find **length** `n` and **tail**.
- Compute effective rotations:

```
k %= n;
```

- Make list **circular**:

```
tail->next = head;
```

- New tail = `(n - k - 1)` th node.
- New head = `newTail->next`.
- Break the circle.

Edge Cases

- Empty list / single node.
- `k == 0`.
- `k % n == 0` → no rotation.
- `k > n`.

Interview Tips

- **Always do** `k %= n` first.
- Circular list simplifies pointer handling.
- New head is `n-k` **th node**, new tail is one node before it.

Code Snippet

```
ListNode* rotateRight(ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;

    int n = 1;
    ListNode* tail = head;
    while (tail->next) {
        tail = tail->next;
        n++;
    }

    k %= n;
    if (k == 0) return head;

    tail->next = head;   // Make circular

    ListNode* newTail = head;
    for (int i = 1; i < n - k; i++)
        newTail = newTail->next;

    ListNode* newHead = newTail->next;
    newTail->next = nullptr;

    return newHead;
}
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

Pattern to Remember

```
Find Length
     ↓
k %= n
     ↓
Make Circular
     ↓
Find New Tail
     ↓
Break Circle
```